

Chapter 60: Memory Leaks

Scenario 1: The SPA That Never Fully Reloads

Situation: A logistics dashboard is left open on a warehouse floor kiosk for entire shifts. Support tickets report the browser tab becoming sluggish after 4-6 hours, eventually crashing with "Aw, Snap!" in Chrome. Users navigate between a dozen routes (orders, inventory, shipping, reports) all day without ever refreshing the tab.

Question: How would you investigate this and narrow down which part of the app is responsible for unbounded memory growth?

Answer: Start with reproducing the growth under controlled conditions rather than guessing:

1. Open Chrome DevTools → Performance Monitor to watch "JS heap size" live while navigating between routes repeatedly (e.g., Orders → Inventory → Orders → Inventory, 20 times). If heap trends upward and never comes back down after forced GC, that's a leak signature, not just normal fluctuation.
2. Take a heap snapshot (Memory tab → Heap snapshot) on the Orders route, navigate away, force GC (the trash-can icon), navigate back, force GC again, and take a second snapshot. Use "Comparison" view between the two snapshots.
3. Look at the "# Delta" and "Retained Size" columns sorted by constructor name. Detached DOM trees, growing arrays of subscriptions, or accumulating component instances that should have been destroyed are the tell-tale signs. A common finding: `OrdersComponent` retained count increases by 1 every time you visit the route, even though the router should have destroyed the previous instance.
4. Right-click a retained `OrdersComponent` instance → "Reveal in Retainers" to see the retainer chain. Frequently the culprit is a singleton service (root-provided) holding a reference via a subscription (`service.data$.subscribe(...)` never unsubscribed) or a manually registered callback (`window.addEventListener`, `ResizeObserver`, third-party chart library instance).
5. Once the retainer path is identified (e.g., a root `NotificationService` keeps an array of callbacks registered by each component instance and never removes them), fix at the source — every subscription/registration created in a component must be torn down in that component's destroy lifecycle.

Fix pattern using `DestroyRef` (works in both constructor-injection contexts and functional contexts, no need for `OnDestroy` boilerplate):

```
import { Component, inject, DestroyRef } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { NotificationService } from './notification.service';

@Component({
  selector: 'app-orders',
  standalone: true,
  template: `...`,
})
export class OrdersComponent {
  private notificationService = inject(NotificationService);
  private destroyRef = inject(DestroyRef);
```

```

constructor() {
  this.notificationService.messages$
    .pipe(takeUntilDestroyed(this.destroyRef))
    .subscribe(msg => this.handleMessage(msg));
}

private handleMessage(msg: string) {
  // ...
}
}

```

Verification: repeat the snapshot-diff cycle (visit → GC → snapshot, revisit → GC → snapshot) 20 times. The retained count of `OrdersComponent` and its child directives should stay flat (0 growth) after the fix, and the Performance Monitor heap line should plateau instead of climbing.

Interviewer intent: Tests whether the candidate has an actual DevTools-driven methodology for leak-hunting rather than "add `takeUntilDestroyed` everywhere and hope," and whether they understand retainer-chain analysis.

Scenario 2: Dashboard Widget Leaking After Repeated Open/Close

Situation: A BI dashboard lets users drag-add "widget" components (charts, KPI tiles) onto a canvas and remove them via an "X" button, which calls `viewContainerRef.clear()` on the widget's dynamic host. QA reports that after adding and removing the same chart widget 50 times, the tab's memory grows by roughly 150 MB and never drops, even though the canvas visually looks empty.

Question: Why does removing a dynamically created component from the DOM not free its memory, and how do you fix it?

Answer: `viewContainerRef.clear()` (or `.remove()`) does destroy the Angular component instance and removes its DOM node, which correctly triggers `ngOnDestroy`. The leak, therefore, is virtually never "Angular didn't destroy the component" — it's that something *external* to Angular's change detection still holds a reference into the destroyed instance's scope, preventing GC:

- A charting library (Chart.js, ECharts, ApexCharts) instance created in `ngOnInit` via `new Chart(ctx, config)` is never explicitly destroyed. The chart library internally attaches DOM event listeners (mousemove, resize) and keeps references to the canvas element and config objects, which in turn close over `this` (the component). Even though Angular has destroyed the component's logical view, the chart library's internal registry (many chart libs keep a static array/map of all instances for reflow-on-resize) still holds a strong reference.
- Confirm via heap snapshot: filter constructor list for the chart library's internal class (e.g., `Chart`) — count should equal number of widgets ever created, not currently active widgets. "Retainers" view will show a path like `Chart.instances[] → chart → ctx.canvas → __ngContext__ → WidgetComponent`.

Fix — always dispose third-party libraries explicitly in `ngOnDestroy`, and prefer wiring it up via `DestroyRef.onDestroy()` so the cleanup lives next to creation:

```

import { Component, ElementRef, inject, DestroyRef, viewChild, afterNextRender } from
'@angular/core';
import { Chart } from 'chart.js/auto';

@Component({
  selector: 'app-chart-widget',
  standalone: true,
  template: `<canvas #canvasRef></canvas>`,
})
export class ChartWidgetComponent {
  private canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>('canvasRef');
  private destroyRef = inject(DestroyRef);
  private chart?: Chart;

  constructor() {
    afterNextRender(() => {
      this.chart = new Chart(this.canvasRef().nativeElement, {
        type: 'line',
        data: { datasets: [] },
      });

      this.destroyRef.onDestroy(() => {
        this.chart?.destroy(); // releases internal registry reference & listeners
        this.chart = undefined;
      });
    });
  }
}

```

Verification: repeat add/remove 50 times with GC + heap snapshot between batches of 10. The `Chart` constructor count in the snapshot should stay at (or near) the number of *currently open* widgets, not the cumulative total ever created. Also confirm the "Detached HTMLCanvasElement" count doesn't grow.

Interviewer intent: Checks whether the candidate understands that Angular destroying a component is necessary but not sufficient — third-party imperative libraries need explicit disposal, and framework cleanup ≠ full cleanup.

Scenario 3: WebSocket Connection Surviving Component Destroy

Situation: A live trading ticker component opens a WebSocket connection in `ngOnInit`/constructor to stream price updates. Users switch between the "Ticker" tab and "Portfolio" tab frequently. After 30 minutes of tab-switching, the Network tab in DevTools shows a growing number of simultaneous open WS connections (visible under WS filter), and the server team reports many stale connections still sending data to a client that's no longer listening.

Question: The component is destroyed when the user navigates away — why is the socket still open, and how do you guarantee cleanup?

Answer: The likely root cause is that the socket was created directly (`new WebSocket(url)`) with `.onmessage` callbacks referencing component state, but `.close()` was never called anywhere — the component assumed garbage collection would somehow tear down the connection, but a live, open WebSocket is a strong root from the browser's networking stack (not from JS reachability) and will keep sending messages, and the `onmessage` closure keeps the destroyed component instance reachable from GC's perspective too (a genuine dual leak: network resource + JS memory).

Diagnosis steps:

1. Network tab, filter "WS", observe count of connections climbing as user switches tabs — confirms connections aren't closing on navigation.
2. Heap snapshot comparison shows retained `TickerComponent` instances proportional to number of visits; retainer chain shows `WebSocket.onmessage` closure → component.

Fix — wrap the socket lifecycle in an Observable and rely on `takeUntilDestroyed`, or explicitly close it in a `DestroyRef.onDestroy` hook. Prefer the Observable wrapper since it composes with RxJS operators (retry, backoff) and unsubscription automatically triggers cleanup via a teardown function:

```
import { Component, inject, DestroyRef } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { Observable } from 'rxjs';

function websocket$(url: string): Observable<MessageEvent> {
  return new Observable<MessageEvent>(subscriber => {
    const socket = new WebSocket(url);
    socket.onmessage = (event) => subscriber.next(event);
    socket.onerror = (err) => subscriber.error(err);
    socket.onclose = () => subscriber.complete();

    // Teardown: called on unsubscribe, i.e. when takeUntilDestroyed fires
    return () => {
      if (socket.readyState === WebSocket.OPEN || socket.readyState ===
WebSocket.CONNECTING) {
        socket.close();
      }
    };
  });
}

@Component({ selector: 'app-ticker', standalone: true, template: `...` })
export class TickerComponent {
  private destroyRef = inject(DestroyRef);

  constructor() {
    websocket$('wss://prices.example.com/stream')
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(event => this.updatePrice(JSON.parse(event.data)));
  }

  private updatePrice(data: unknown) { /* ... */ }
}
```

Verification: switch tabs 20 times, watch the Network WS filter — the open-connection count should stay at 0 or 1 (only the currently active tab's socket), never accumulating. Confirm server-side connection count logs drop to match. Heap snapshot retained `TickerComponent` count should return to 0 after navigating away + GC.

Interviewer intent: Distinguishes candidates who understand that memory leaks and resource leaks (sockets, timers, file handles) are related but separate problems — both need explicit teardown, and Observable teardown functions are the idiomatic Angular/RxJS way to unify them.

Scenario 4: Renderer2 Event Listener Never Removed

Situation: A component uses `Renderer2.listen(document, 'click', ...)` to implement a "click outside to close" dropdown. It's used in a table with hundreds of rows, each row instantiating this dropdown component on demand (e.g., a context menu). After scrolling through a large virtualized table for a while, DevTools' "Event Listeners" tab on the `document` node shows thousands of click listeners still attached.

Question: `Renderer2.listen` returns something — what's being missed, and what's the correct pattern?

Answer: `Renderer2.listen(target, eventName, callback)` returns an **unlisten function**, not a subscription-like object with implicit cleanup. If that returned function is discarded (not stored and called), the listener is permanently attached to `document` — and because it's a closure, it keeps the component instance (and its whole view) alive as long as `document` exists, i.e., for the entire lifetime of the app.

Diagnosis: Elements panel → click on `<html>` or `<body>` in Elements tab → Event Listeners sub-tab → expand "click" → count grows with every dropdown ever instantiated (not just currently open ones). Heap snapshot retainer path: `document` → (internal listener list) → closure → `DropdownComponent`.

Fix — always capture and invoke the unlisten function, ideally tied to `DestroyRef`:

```
import { Component, Renderer2, ElementRef, inject, DestroyRef } from '@angular/core';

@Component({
  selector: 'app-row-dropdown',
  standalone: true,
  template: `...`,
})
export class RowDropdownComponent {
  private renderer = inject(Renderer2);
  private elementRef = inject(ElementRef<HTMLElement>);
  private destroyRef = inject(DestroyRef);

  open() {
    const unlisten = this.renderer.listen('document', 'click', (event: MouseEvent) => {
      if (!this.elementRef.nativeElement.contains(event.target as Node)) {
        this.close();
      }
    });
  }

  // Remove the listener either when the dropdown closes OR the component is destroyed,
  // whichever comes first.
  this.destroyRef.onDestroy(unlisten);
}
```

```
}

close() {
  // hide dropdown UI
}
}
```

For a more robust pattern (closing the dropdown should also remove the listener immediately, not wait for destroy), store `unlisten` in a field and call it from `close()` too, guarding against double-invocation.

Verification: instantiate and close 100 dropdowns; check Elements → Event Listeners on `document` — the "click" listener count attributable to this feature should return to 0 (or 1, if one is currently open), not accumulate to 100.

Interviewer intent: Tests whether the candidate knows the exact API contract of `Renderer2.listen` (return value is the cleanup function) — a very common real-world bug because the return value is easy to ignore since `listen()` doesn't require capturing it to compile.

Scenario 5: Singleton Service Holding References to Destroyed Components

Situation: A root-provided `AnalyticsService` exposes a method `registerComponent(ref: MyComponent)` so it can call `ref.refreshMetrics()` on a timer. Over a day of usage with many short-lived components registering themselves, a heap snapshot shows an internal array in `AnalyticsService` with thousands of entries, most pointing to destroyed component instances.

Question: Why does a `providedIn: 'root'` service holding component references cause a leak, and what's the fix?

Answer: A root singleton service lives for the entire application lifetime. If it stores strong references to components in a mutable array/map and only ever pushes (never removes on destroy), every component that ever called `registerComponent` remains reachable via that array forever — regardless of whether Angular destroyed the component's view. This is a classic "long-lived object retains short-lived object" leak, distinguishable from the earlier scenarios because the retainer root is application-level state, not the DOM or a timer/socket.

Diagnosis: heap snapshot → search constructor `AnalyticsService` → expand `_components` (or whatever internal field) array → observe length far exceeding the number of currently rendered components. Retainer view for one of the destroyed component entries confirms it's only reachable through this array (Detached DOM tree checkbox will also be true for its root element, since it's fully removed from the visible DOM but retained in JS memory).

Fix options, in order of preference:

1. **Best:** avoid the registration pattern altogether — have the service expose an `Observable / Signal` that components read from, rather than the service calling back into components.
2. **If registration is unavoidable** (e.g., legacy interop), require unregistration paired with `DestroyRef`, or use a `WeakSet / WeakRef` so the GC can reclaim entries even if unregistration is missed (defense in depth — the primary fix is still explicit unregistration).

```
import { Injectable } from '@angular/core';

interface Refreshable {
  refreshMetrics(): void;
}

@Injectable({ providedIn: 'root' })
export class AnalyticsService {
  // weakSet does not prevent GC of components that lose all other references,
  // and iteration order/removal semantics are fine for a "notify all" use case.
  private components = new Set<Refreshable>();

  register(component: Refreshable): () => void {
    this.components.add(component);
    return () => this.components.delete(component); // caller MUST invoke on destroy
  }

  refreshAll() {
    for (const c of this.components) c.refreshMetrics();
  }
}
```

```
import { Component, inject, DestroyRef } from '@angular/core';
import { AnalyticsService } from '../analytics.service';

@Component({ selector: 'app-widget', standalone: true, template: `...` })
export class WidgetComponent {
  private analytics = inject(AnalyticsService);
  private destroyRef = inject(DestroyRef);

  constructor() {
    const unregister = this.analytics.register(this);
    this.destroyRef.onDestroy(unregister);
  }

  refreshMetrics() { /* ... */ }
}
```

Verification: mount/destroy 200 `WidgetComponent` instances, then heap-snapshot and inspect `AnalyticsService.components` — its size should equal the number of currently mounted widgets (e.g., 0 if none are mounted), not 200.

Interviewer intent: Probes understanding of ownership direction — services outliving components must never be the ones holding strong references without a matching unregister path; this is one of the most common real-world Angular leak patterns in dashboards/plugins architectures.

Scenario 6: setInterval Polling Never Cleared

Situation: A "live status" badge component polls a REST endpoint every 5 seconds via `setInterval` to show green/red health. Support notices that after leaving the app open overnight with the badge on 15 different pages visited throughout the day, network traffic to the health endpoint is 15x higher than expected — as if every previous badge instance is still polling.

Question: Diagnose and fix.

Answer: Classic timer leak: `setInterval` returns a handle; if `clearInterval` is never called, the callback keeps firing forever, and — critically — the callback closure keeps `this` (the component) alive, so it's both a network-traffic leak and a memory leak simultaneously.

Diagnosis: Network tab, filter by the health endpoint URL, observe multiple requests firing at the same near-identical timestamps (e.g., 15 requests every 5 seconds instead of 1) — a strong signal that multiple old interval timers are still active. Confirm in heap snapshot: search for the component constructor, see retained count matching number of times the badge was ever rendered.

Fix — never use raw `setInterval` in components; wrap in RxJS `interval/timer` and use `takeUntilDestroyed`, which composes cleanly and is testable with marble testing:

```
import { Component, inject, DestroyRef, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { interval, switchMap } from 'rxjs';
import { HttpClient } from '@angular/common/http';

@Component({ selector: 'app-status-badge', standalone: true, template: `{{ status() }}` })
export class StatusBadgeComponent {
  private http = inject(HttpClient);
  private destroyRef = inject(DestroyRef);
  status = signal<'up' | 'down'>('up');

  constructor() {
    interval(5000)
      .pipe(
        switchMap(() => this.http.get<{ ok: boolean }>('/api/health')),
        takeUntilDestroyed(this.destroyRef),
      )
      .subscribe(res => this.status.set(res.ok ? 'up' : 'down'));
  }
}
```

If `setInterval` must be used directly (e.g., no RxJS in a tiny widget), store the handle and clear it explicitly:

```
constructor() {
  const handle = setInterval(() => this.poll(), 5000);
  this.destroyRef.onDestroy(() => clearInterval(handle));
}
```


Verification: navigate to the badge page 10 times over a few minutes (with time between visits), then watch Network tab for 30 seconds — request count to the health endpoint should match the number of *currently mounted* badges (typically 1), not 10.

Interviewer intent: Basic but frequently missed — checks that the candidate reflexively pairs every `setInterval` / `setTimeout` with a teardown, and knows the RxJS-idiomatic replacement.

Scenario 7: @Input-Bound Third-Party Map Widget Not Disposed

Situation: A property-listing app embeds a Leaflet/Google Maps map per listing card. Users scroll through an infinite-scroll list of 200+ listings; as cards scroll out of view they're removed via `*ngIf`/virtual scroll recycling. After scrolling through the full list, the tab uses over 1 GB of memory, and `performance.memory.usedJSHeapSize` (via `chrome://` or the Memory tab) never drops even after minutes of idle time.

Question: How do you find which part of the map integration is leaking, and how do you fix it given the map library manages its own DOM/canvas internally?

Answer: Map libraries (Leaflet, Mapbox GL, Google Maps JS API) attach a large number of internal event listeners (drag, zoom, resize, tile-load) and keep tile image caches. If the component only removes the DOM container without calling the library's own `.remove()` / `.destroy()` API, the library's internal registries keep everything reachable.

Diagnosis:

1. Heap snapshot comparison after scrolling through 50 cards: filter for the map library's internal classes (e.g., `L.Map`, `google.maps.Map` isn't directly enumerable, but its internal `MapCanvasProjection`/tile layer objects are). Growing counts confirm no disposal.
2. "Detached DOM tree" filter in the snapshot: look for detached `<div class="leaflet-container">` nodes — these are map containers removed from the visible DOM but still retained because the library's internal handlers reference them.

Fix — call the library's teardown API in `ngOnDestroy` / `DestroyRef.onDestroy`, and do it defensively even for `*ngIf`-toggled components since Angular will destroy the component but not the imperative library instance:

```
import { Component, ElementRef, ViewChild, inject, DestroyRef, afterNextRender } from
 '@angular/core';
import * as L from 'leaflet';

@Component({
  selector: 'app-listing-map',
  standalone: true,
  template: `<div #mapEl class="map-container"></div>`,
})
export class ListingMapComponent {
  private mapEl = ViewChild.required<ElementRef<HTMLDivElement>>('mapEl');
  private destroyRef = inject(DestroyRef);
  private map?: L.Map;
```

```

constructor() {
  afterNextRender(() => {
    this.map = L.map(this.mapEl().nativeElement).setView([37.7749, -122.4194], 13);
    L.tileLayer('https://tile.example.com/{z}/{x}/{y}.png').addTo(this.map);

    this.destroyRef.onDestroy(() => {
      this.map?.remove(); // Leaflet's own cleanup: removes listeners, tile cache, DOM
      this.map = undefined;
    });
  });
}
}

```

For virtual-scroll (CDK `cdk-virtual-scroll-viewport`), confirm the recycling strategy actually destroys the component (rather than just hiding it via CSS) — some custom virtual scroll implementations reuse component instances and merely rebind `@Input()`s, in which case the map is never re-created/destroyed at all and this whole class of bug doesn't apply, but then stale map state becomes a correctness bug instead.

Verification: scroll through the full 200-item list twice, forcing GC between passes; heap snapshot should show detached map-container DOM nodes returning to 0, and `usedJSHeapSize` should stabilize rather than growing linearly with scroll distance.

Interviewer intent: Tests awareness that Angular's `*ngIf`/virtual-scroll destroying a component wrapper does not automatically dispose of imperative third-party widgets that manage their own DOM subtree and internal listener registries.

Scenario 8: Detached DOM Trees From Manual DOM Manipulation

Situation: A rich-text editor component uses direct DOM manipulation (`document.createElement`, manual `appendChild`) for a custom toolbar overlay, bypassing Angular's template/renderer entirely for performance reasons. After opening and closing the editor 30 times, a heap snapshot shows dozens of "Detached HTMLDivElement" nodes retained, forming trees of 10-20 nodes each.

Question: What causes detached DOM trees specifically (as opposed to a live leak), and how do you eliminate them?

Answer: A "detached DOM tree" in a heap snapshot means a DOM node has been removed from `document` (not visible, not part of the render tree) but is still reachable from JavaScript — usually because a JS variable, closure, or event listener still references it. Chrome cannot garbage-collect it despite it having zero rendering cost, because reachability (not visibility) drives GC.

Root cause here: the component stores toolbar DOM references in instance fields (e.g., `private toolbarEl: HTMLDivElement`) and/or attaches raw `addEventListener` calls directly to those elements, but on close only removes the toolbar from its **visual parent** (e.g., `toolbarEl.remove()` on a child, but the component itself still holds `this.toolbarEl`, and any `addEventListener`-registered callback closures keep the whole chain alive if the listener is on `window`/`document` rather than the toolbar itself).

Diagnosis: Heap snapshot → check "Detached HTMLDivElement (or similar)" under Summary view, sorted by retained size → Reveal in Retainers → trace to the owning component instance's field or an external listener.

Fix — null out fields on destroy, and always target listeners on the narrowest possible element (attach to the toolbar itself, not `window`, when semantically valid) so removing the toolbar from the DOM and clearing the reference is sufficient:

```
import { Component, inject, DestroyRef, afterNextRender, ElementRef, ViewChild } from
 '@angular/core';

@Component({
  selector: 'app-rich-editor',
  standalone: true,
  template: `<div #hostEl></div>`,
})
export class RichEditorComponent {
  private hostEl = ViewChild.required<ElementRef<HTMLDivElement>>>('hostEl');
  private destroyRef = inject(DestroyRef);
  private toolbarEl?: HTMLDivElement;
  private onToolbarClick = (e: MouseEvent) => this.handleToolbarClick(e);

  constructor() {
    afterNextRender(() => {
      this.toolbarEl = document.createElement('div');
      this.toolbarEl.className = 'rte-toolbar';
      this.toolbarEl.addEventListener('click', this.onToolbarClick);
      this.hostEl().nativeElement.appendChild(this.toolbarEl);

      this.destroyRef.onDestroy(() => {
        this.toolbarEl?.removeEventListener('click', this.onToolbarClick);
        this.toolbarEl?.remove();
        this.toolbarEl = undefined; // release the last JS reference
      });
    });
  }

  private handleToolbarClick(e: MouseEvent) { /* ... */ }
}
```

Verification: open/close the editor 30 times, force GC, take a heap snapshot, and search Summary view for "Detached" — count attributable to the toolbar should be 0. Where feasible, prefer Angular's own `template/Renderer2` for such DOM instead of raw manipulation, since Angular guarantees teardown of elements it created.

Interviewer intent: Verifies the candidate understands the precise mechanics of "detached DOM tree" (reachability vs. visibility) and knows this is distinct from — but related to — event listener leaks.

Scenario 9: RxJS Subject Never Completed in a Shared Service

Situation: A `SearchService` (root-provided) exposes a `Subject<string>` for search-term changes that multiple filter components subscribe to via `.pipe(debounceTime(300), switchMap(...))`. As users navigate in and out of the search page repeatedly, a memory profiling session shows the number of active subscriptions to this subject climbing indefinitely (visible by adding an internal counter or via RxJS devtools).

Question: The `Subject` itself lives in a root singleton and won't be destroyed — so what's leaking, and how do you avoid subscriber accumulation?

Answer: The `Subject` instance is fine to live forever — it's a hot observable meant to be long-lived. The leak is on the *subscriber* side: each `FilterComponent` instance subscribes to `searchService.term$` but never unsubscribes, so every past component instance's subscription callback remains registered on the subject's internal observer list, keeping every past `FilterComponent` reachable via the subject → its closures → the destroyed component.

This is conceptually identical to Scenario 5 (singleton retaining component references) but manifests through RxJS's internal subscriber list rather than an app-level array — worth explicitly calling out in an interview because candidates often think "the service isn't destroyed, so this is expected" and stop investigating, when the real issue is subscriber cleanup, not subject lifetime.

Diagnosis: heap snapshot, find the `Subject` instance, expand `observers` (or `_subscribers` depending on RxJS version) array field — its length should equal the number of currently mounted filter components, not the cumulative total ever mounted.

Fix, standard `takeUntilDestroyed`:

```
import { Component, inject, DestroyRef } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { SearchService } from '../search.service';
import { debounceTime, switchMap } from 'rxjs';

@Component({ selector: 'app-filter', standalone: true, template: `...` })
export class FilterComponent {
  private searchService = inject(SearchService);
  private destroyRef = inject(DestroyRef);

  constructor() {
    this.searchService.term$
      .pipe(
        debounceTime(300),
        switchMap(term => this.searchService.search(term)),
        takeUntilDestroyed(this.destroyRef),
      )
      .subscribe(results => this.render(results));
  }

  private render(results: unknown[]) { /* ... */ }
}
```

Verification: mount/destroy `FilterComponent` 25 times, then inspect

`searchService.term$['observers'].length` (or use RxJS DevTools extension) — should equal current mount count (0 or 1), not 25.

Interviewer intent: Distinguishes "the Subject is a singleton so it's fine" (wrong reasoning) from correctly identifying that subscriber accumulation, not subject lifetime, is the actual leak — a subtlety many mid-level candidates miss.

Scenario 10: @HostListener on window Accumulating Across Route Changes

Situation: A `ScrollSpyDirective` applied to many section components uses `@HostListener('window:scroll')` to highlight the active nav item. The app has a single-page marketing site with route-based tabs; each tab mounts several `ScrollSpyDirective` instances. After tabbing through the site 10 times, scrolling becomes visibly janky (frame drops in the Performance panel's "Scripting" band during scroll).

Question: Does `@HostListener('window:...')` get cleaned up automatically by Angular, and if this is still leaking, what would you check?

Answer: Good news first: `@HostListener` bound to `window / document / body` **is** automatically cleaned up by Angular — the framework's `DomRenderer` registers these via `Renderer2.listen` internally and stores the `unlisten` function, calling it when the host directive/component is destroyed. So if directives are genuinely destroyed on route change, this specific API is not the leak source.

This means the real investigation should go one level up: is the directive actually being destroyed on navigation? Common causes it might not be:

- The directive is on an element inside a component that's kept alive by route reuse strategies (custom `RouteReuseStrategy` that caches routes for back/forward navigation) — the component (and its directives) are detached from the router outlet but never destroyed, by design, so `@HostListener` handlers pile up because each cached route still has live listeners even while not displayed.
- The site uses `<router-outlet>` inside an `*ngIf`-guarded wrapper with `[hidden]` instead of actually removing components — visually hidden but still in the component tree and still receiving scroll events, doing wasted work (a performance leak even without a strict memory leak).

Diagnosis: Performance panel → record a scroll while on tab 1 after having visited tabs 1-10 → the "Scripting" flame chart shows scroll-handler callbacks firing for section components belonging to tabs the user is no longer on. Cross-check by adding a temporary `console.count('scrollspy-tick')` per directive instance (or, better, Angular DevTools' component tree — inspect whether previous tabs' components still show up in the tree).

Fix — if a custom `RouteReuseStrategy` is intentionally caching routes, that's a deliberate trade-off and each cached component must explicitly pause its own listeners (e.g., via a lifecycle hook triggered by the reuse strategy) rather than relying on destroy-based cleanup, since destroy never happens for cached routes:

```
import { Directive, inject, DestroyRef, HostListener } from '@angular/core';

@Directive({ selector: '[appScrollSpy]', standalone: true })
export class ScrollSpyDirective {
```

```

private active = true; // toggled by a route-activation hook when using route caching

@HostListener('window:scroll')
onScroll() {
  if (!this.active) return; // guard against work while route is cached/inactive
  // ... highlight logic
}

setActive(active: boolean) {
  this.active = active;
}
}

```

If route caching is not intentional, the actual fix is to stop caching this particular route (adjust `RouteReuseStrategy.shouldDetach()` to return `false` for it) so components are properly destroyed and Angular's automatic `@HostListener` cleanup takes effect.

Verification: with the guard in place (or reuse strategy fixed), record a Performance trace while scrolling on tab 1 after visiting all 10 tabs — the Scripting band should only show handler invocations from tab 1's directive instances.

Interviewer intent: Tests the nuance that `@HostListener` is auto-cleaned-up by Angular (so candidates shouldn't blame it reflexively) and pushes them to reason about `RouteReuseStrategy` as a less obvious cause of "destroy never happens."

Scenario 11: Angular Signals `effect()` Leaking via Untracked Closures

Situation: A component uses `effect()` to sync a signal's value into a third-party non-Angular widget (e.g., a jQuery-based legacy tooltip library) each time it changes. After the component is destroyed and recreated many times (e.g., inside a modal opened/closed repeatedly), a heap snapshot shows growing numbers of the tooltip library's internal objects.

Question: `effect()` created inside a component's injection context is supposed to be tied to that component's lifecycle — why would this still leak?

Answer: `effect()` created in a constructor (or field initializer) within a component's own injection context is automatically destroyed when the component is destroyed — Angular ties the effect's lifecycle to the enclosing `DestroyRef` by default. So if the leak persists, the likely causes are:

1. The `effect()` was created with `{ manualCleanup: true }` (or the older `{ injector }` override pointing to a longer-lived injector, e.g., the root `EnvironmentInjector`) without a corresponding manual `.destroy()` call — this opts out of automatic tie-in to the component.
2. The effect's callback body itself creates the leak independent of the effect's own lifecycle — e.g., it calls `thirdPartyTooltip.attach(e1)` on every run but never calls the corresponding `.detach()` / `.destroy()` for the *previous* attachment, so even though the effect stops firing after destroy, the last-created tooltip instance (and every one before it that wasn't cleaned up) remains attached to a detached DOM node forever.

Diagnosis: check the effect's creation call for `manualCleanup` or a custom `injector` option first (quick code review catches this immediately). If absent, heap-snapshot the tooltip library's internal registry (many attach a `data-*` attribute + `WeakMap`/plain object keyed by DOM element) — if it's a plain object/Map instead of `WeakMap`, it will retain the tooltip instance (and, transitively, the DOM element/component) forever.

Fix:

```
import { Component, effect, inject, DestroyRef, ElementRef, ViewChild, signal } from
'@angular/core';
import tooltip from 'legacy-tooltip-lib';

@Component({ selector: 'app-modal-field', standalone: true, template: `<input #inputEl>` })
export class ModalFieldComponent {
  private inputEl = ViewChild.required<ElementRef<HTMLInputElement>>('inputEl');
  private destroyRef = inject(DestroyRef);
  hint = signal('Enter a value');
  private currentTooltip?: { destroy(): void };

  constructor() {
    // Default effect() lifecycle IS tied to this component – no manualCleanup, no custom
    injector.
    effect(() => {
      this.currentTooltip?.destroy(); // clean up previous attachment every
run
      this.currentTooltip = tooltip.attach(this.inputEl().nativeElement, this.hint());
    });

    this.destroyRef.onDestroy(() => this.currentTooltip?.destroy()); // final cleanup on
destroy
  }
}
```

Verification: open/close the modal 40 times, heap-snapshot, and check the tooltip library's internal instance count — should equal 0 (or 1, if modal is open), not 40. Also grep the codebase for `manualCleanup: true` and `effect(..., { injector: ... })` as a quick audit for effects that opted out of automatic cleanup.

Interviewer intent: Tests up-to-date knowledge of Signals API lifecycle semantics (effects auto-tie to `DestroyRef` unless explicitly opted out) versus the older mental model where all cleanup had to be manual — a good discriminator for candidates who've kept up with recent Angular versions.

Scenario 12: Zone.js Patched Timers Retaining Injector Context

Situation: A notifications component sets a `setTimeout` to auto-dismiss a toast after 5 seconds. Under heavy toast usage (e.g., a bulk-import feature that fires 200 toasts in a few seconds), DevTools' Performance Monitor shows heap climbing sharply and not fully recovering, correlating exactly with toast bursts.

Question: Each toast auto-dismisses after 5s and its DOM is removed — so why would memory still grow disproportionately to the number of *currently visible* toasts?

Answer: Two contributing causes are common here, and a good answer distinguishes them:

1. **Legitimate but temporary:** 200 outstanding `setTimeout` callbacks are, briefly, all legitimately alive and each closes over its toast component/data — this is expected and not a "leak" as long as they all fire and clean up within the 5s window. The fix here (if this alone were the issue) is just to confirm it drains — heap should return to baseline within ~5-6 seconds after the burst, not stay elevated.
2. **Actual leak:** if the toast component's `ngOnDestroy` cancels the timer only when the *user* manually dismisses it, but the auto-dismiss path (`setTimeout` firing naturally) doesn't also clear an *outer* reference — e.g., a `ToastService` keeps an array of `{ id, componentRef }` and only splices it on manual dismiss, never on auto-dismiss firing — then every auto-dismissed toast's `componentRef` (and its destroyed-but-still-referenced component) remains in the service's array forever. This is the same "singleton array never pruned" pattern as Scenario 5, applied to a timer-driven removal path specifically.

Diagnosis: heap-snapshot after a burst + waiting well past the 5s window (say, 30s, with GC forced) — if heap hasn't returned near baseline, inspect `ToastService`'s internal array length vs. actual number of visible toasts in the DOM. A mismatch (array length ~200, visible toasts ~0) confirms cause #2.

Fix — ensure the removal path is unified so both manual dismiss and auto-dismiss go through the same cleanup function:

```
import { Injectable, ApplicationRef, createComponent, EnvironmentInjector, inject } from
 '@angular/core';
import { ToastComponent } from './toast.component';

interface ToastEntry {
  id: number;
  ref: ReturnType<typeof createComponent<ToastComponent>>;
  timeoutHandle: ReturnType<typeof setTimeout>;
}

@Injectable({ providedIn: 'root' })
export class ToastService {
  private appRef = inject(ApplicationRef);
  private envInjector = inject(EnvironmentInjector);
  private toasts: ToastEntry[] = [];
  private nextId = 0;

  show(message: string) {
    const id = this.nextId++;
    const ref = createComponent(ToastComponent, { environmentInjector: this.envInjector });
    ref.instance.message = message;
    document.body.appendChild(ref.location.nativeElement);
    this.appRef.attachView(ref.hostView);

    const timeoutHandle = setTimeout(() => this.dismiss(id), 5000);
    this.toasts.push({ id, ref, timeoutHandle });
  }

  // Single cleanup path used by BOTH auto-dismiss and manual dismiss.
  dismiss(id: number) {
    const index = this.toasts.findIndex(t => t.id === id);
    if (index === -1) return;
```



```

    const [entry] = this.toasts.splice(index, 1); // remove from tracking array
    unconditionally
    clearTimeout(entry.timeoutHandle);
    this.appRef.detachView(entry.ref.hostView);
    entry.ref.destroy();
  }
}

```

Verification: fire 200 toasts, wait 10 seconds past the last auto-dismiss, force GC, heap-snapshot — `ToastService.toasts.length` should be 0 and `ToastComponent` retained instance count should be 0.

Interviewer intent: Tests the ability to distinguish transient, expected memory usage (in-flight timers) from a genuine leak (a tracking array not pruned on every removal path), which requires reasoning about *all* code paths that remove an item, not just the obvious one.

Scenario 13: forRoot() Singleton Service Accumulating Event History

Situation: A `LoggingService`, provided via `providedIn: 'root'`, buffers the last N app events for a "recent activity" debug panel by pushing to an array on every user action (clicks, route changes, form submits). Over a multi-hour session, heap snapshots show this array growing unbounded (tens of thousands of entries), each holding references to event payloads that include DOM elements (`event.target`).

Question: This isn't a component-destroy problem at all — what's leaking and why is it worse than a typical component leak?

Answer: This is a different category from the previous scenarios: there's no component-lifecycle bug here — the singleton is behaving exactly as designed (append-only), but the design itself is the leak. It's worse than typical component leaks because:

- It grows monotonically with *user activity*, not with a specific buggy component, so it's harder to isolate to "one screen" — QA might not reproduce it on any single flow.
- Storing raw DOM references (`event.target`) inside the log entries means every logged event pins an entire DOM subtree in memory — even elements that have been removed from the page stay retained as "detached" trees, and worse, propagate up the ancestor chain (a `Node` retains its `parentNode`, so pinning a leaf `<span>` can pin most of an old page's tree depending on how the DOM reference graph is structured — actually in the DOM, children reference parents via `parentNode`, and parents reference children via `childNodes`, so retaining any single node the array holds can, via bidirectional traversal in the retainer graph, prevent collection of the whole subtree it belonged to).

Diagnosis: heap snapshot → search constructor `LoggingService` → inspect the buffer array length (unbounded) and "Retained Size" column — likely one of the largest retainers in the whole snapshot. Filter Summary view by "Detached" to see how many entries are pinning subtrees that are otherwise fully removed from the page.

Fix — apply a hard cap (ring buffer) and never store live object/DOM references in log entries, only serializable, minimal data:

```

import { Injectable } from '@angular/core';

```

```

interface LogEntry {
  timestamp: number;
  type: string;
  targetSelector: string; // serialized description, NOT a live element reference
}

const MAX_ENTRIES = 500;

@Injectable({ providedIn: 'root' })
export class LoggingService {
  private buffer: LogEntry[] = [];

  log(type: string, target: EventTarget | null) {
    this.buffer.push({
      timestamp: Date.now(),
      type,
      targetSelector: this.describeTarget(target), // extract a string, drop the reference
    });

    if (this.buffer.length > MAX_ENTRIES) {
      this.buffer.splice(0, this.buffer.length - MAX_ENTRIES); // ring-buffer trim
    }
  }

  private describeTarget(target: EventTarget | null): string {
    if (!(target instanceof Element)) return 'unknown';
    return `${target.tagName.toLowerCase()}${target.id ? '#' + target.id : ''}`;
  }

  getRecent(): readonly LogEntry[] {
    return this.buffer;
  }
}

```

Verification: run the app for a simulated multi-hour session (or artificially call `.log()` 100,000 times in a test), then confirm `buffer.length` never exceeds `MAX_ENTRIES` and heap snapshot shows no DOM element retained via `LoggingService`.

Interviewer intent: Tests whether the candidate can identify leaks that are architectural/design flaws (unbounded caches) rather than lifecycle bugs, and understands the danger of storing live DOM/object references versus serialized data in long-lived buffers.

Scenario 14: NgRx / Signal Store Selector Subscription Never Torn Down

Situation: A component subscribes to a global store selector using the imperative `.subscribe()` API directly (instead of the `| async` pipe or `toSignal()`) to imperatively trigger a side effect (e.g., scrolling an element into view) whenever a piece of state changes. After extensive testing, heap snapshots show the component instance count growing with every navigation to the page.

Question: What's different about this from an `@Input`-driven leak, and how would you fix it while keeping the imperative side-effect requirement?

Answer: Directly calling `.subscribe()` on a store selector (whether `NgRx Store.select()`, a signal-store's underlying observable, or an Akita-style query) creates a long-lived subscription against a root-provided store service, structurally identical to Scenario 9/5 — the store's internal subscriber list keeps the component reachable. The distinguishing detail interviewers want to hear addressed: many candidates reach for `| async` in templates as the "safe" default and forget that as soon as you need an imperative side effect (not just binding a value into the template), you're back to manual subscription management.

The idiomatic modern-Angular answer: use `toSignal()` to bridge the observable into a signal, then use `effect()` for the imperative side effect — both are automatically tied to the component's `DestroyRef`, eliminating the manual subscription entirely:

```
import { Component, inject, effect, ElementRef, ViewChild } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { Store } from '@ngrx/store';
import { selectActiveItemId } from './store/selectors';

@Component({ selector: 'app-item-list', standalone: true, template: `<div #listEl>...</div>` })
export class ItemListComponent {
  private store = inject(Store);
  private listEl = ViewChild.required<ElementRef<HTMLDivElement>>>('listEl');

  // Bridges the observable into a signal; subscription lifecycle is tied
  // to the component's injection context automatically.
  private activeItemId = toSignal(this.store.select(selectActiveItemId));

  constructor() {
    effect(() => {
      const id = this.activeItemId();
      if (!id) return;
      const el = this.listEl().nativeElement.querySelector(`[data-id="${id}"]`);
      el?.scrollIntoView({ behavior: 'smooth' });
    });
  }
}
```

If `toSignal()` genuinely doesn't fit (e.g., needing complex operator chains that are easier to express as an Observable pipeline), fall back to `takeUntilDestroyed`:

```
this.store.select(selectActiveItemId)
  .pipe(takeUntilDestroyed()) // implicit DestroyRef lookup works when called in an
  injection context
  .subscribe(id => { /* imperative side effect */ });
```

Verification: navigate to the item list page 30 times, heap-snapshot, and check the store's internal selector subscriber count (NgRx exposes this indirectly via the underlying `BehaviorSubject / Observable` chain, inspectable via Redux DevTools' subscriber count or a heap snapshot on the `Store` instance) — should match currently mounted instances, not cumulative visits.

Interviewer intent: Confirms the candidate knows that `| async / toSignal()` solve the *common* case automatically but that imperative side effects still require deliberate lifecycle management, and tests fluency with `toSignal()` + `effect()` as the modern replacement for manual subscribe-and-sideeffect patterns.

Scenario 15: Global Error Handler Retaining Every Component That Ever Threw

Situation: A custom `ErrorHandler` implementation logs errors along with the component instance for richer debugging context ("attach component state to error reports"). QA reports that after fuzz-testing the app (intentionally triggering errors in many components), overall memory usage is much higher than a normal browsing session, disproportionate to the number of components that should currently be alive.

Question: How would you confirm the `ErrorHandler` is the cause, and what's the safe way to attach debugging context without leaking?

Answer: Confirm via heap snapshot: search for the custom `ErrorHandler` class, expand its internal array/log of captured errors, and check whether entries hold live component instance references (`error.componentInstance = this` pattern) versus serialized snapshots of state. If live references are stored, every component that ever threw an error remains reachable for the entire app session via the `ErrorHandler` singleton (which, like `providedIn: 'root'` services, lives for the app's lifetime) — same underlying pattern as Scenario 5/13, but worth calling out as its own scenario because `ErrorHandler` is a special, easy-to-overlook root-level singleton that many developers don't think of as a "service that can leak."

Fix — capture only serializable, minimal diagnostic data (a snapshot of primitive state or a JSON-safe subset), never a live object/component reference, and cap the log size:

```
import { ErrorHandler, Injectable } from '@angular/core';

interface ErrorLogEntry {
  message: string;
  stack?: string;
  timestamp: number;
  context: Record<string, unknown>; // plain serializable data only
}

const MAX_LOG_ENTRIES = 100;

@Injectable()
export class AppErrorHandler implements ErrorHandler {
  private log: ErrorLogEntry[] = [];

  handleError(error: unknown): void {
    const err = error as Error;

    this.log.push({
      message: err.message ?? String(error),
      stack: err.stack,
      timestamp: Date.now(),
      context: this.safeSnapshot(), // extract plain values, never `this` or a component
    });
  }

  private safeSnapshot(): Record<string, unknown> {
    // ...
  }
}
```

```

});

if (this.log.length > MAX_LOG_ENTRIES) {
  this.log.shift();
}

console.error(error);
}

private safeSnapshot(): Record<string, unknown> {
  return { url: location.href, timestamp: Date.now() };
}
}

```

If component-specific state truly needs to be captured for debugging, serialize the relevant fields (`JSON.parse(JSON.stringify(component.someState))` for plain data, or a hand-written mapper) at the moment of the error rather than storing `component` itself.

Verification: fuzz-test to trigger errors across 50 different component instances, then heap-snapshot and confirm none of those component classes show up as reachable via `AppErrorHandler` (search Retainers from a sampled destroyed component — the error handler's log should not appear in the retainer chain).

Interviewer intent: Tests whether the candidate thinks to audit *infrastructure* singletons (ErrorHandler, logging, analytics) — not just obviously "business logic" services — for the same live-reference anti-pattern, since these cross-cutting services are easy to overlook.

Scenario 16: Intersection/Resize Observer Not Disconnected

Situation: A lazy-loading image gallery uses `IntersectionObserver` per image tile to trigger loading when scrolled into view. The gallery is inside a modal that users open and close repeatedly to browse different albums (each open creates ~100 tiles/observers). After 10 open/close cycles, DevTools' Performance Monitor shows "JS event listeners" (a metric it tracks) climbing by roughly 1,000 each cycle and never dropping.

Question: `IntersectionObserver` doesn't attach via classic `addEventListener` — how do you find this kind of leak, and what's the fix?

Answer: `IntersectionObserver` / `ResizeObserver` / `MutationObserver` instances register internally with the browser's observation machinery, which keeps the observed elements (and the observer's callback closure, and anything the closure references) alive until `.disconnect()` (or `.unobserve()` for the specific target) is called — closing the modal and removing the DOM does not automatically disconnect these observers, unlike some other Web APIs.

Diagnosis: Performance Monitor's "JS event listeners" counter is a reasonable proxy signal (observers register listener-like internal handles) but the more precise tool is a heap snapshot: search for `IntersectionObserver` in the constructor list — count should match currently active tiles, not cumulative tiles ever created. Retainer chain from a stale `IntersectionObserver` instance will lead back to a destroyed `ImageTileComponent`.

Fix — disconnect the observer for each tile in `DestroyRef.onDestroy`, or use a single shared observer at the gallery level with per-tile `unobserve()` calls (more efficient than one observer per tile, and worth mentioning as a secondary optimization):

```

import { Component, ElementRef, viewChild, inject, DestroyRef, afterNextRender, input } from
  '@angular/core';

@Component({
  selector: 'app-image-tile',
  standalone: true,
  template: `<img #imgEl [attr.data-src]="src()" />`,
})
export class ImageTileComponent {
  src = input.required<string>();
  private imgEl = viewChild.required<ElementRef<HTMLImageElement>>('imgEl');
  private destroyRef = inject(DestroyRef);

  constructor() {
    afterNextRender(() => {
      const el = this.imgEl().nativeElement;
      const observer = new IntersectionObserver(entries => {
        if (entries[0].isIntersecting) {
          el.src = this.src();
          observer.unobserve(el);
        }
      });
      observer.observe(el);

      this.destroyRef.onDestroy(() => observer.disconnect());
    });
  }
}

```

For the shared-observer variant (recommended at scale — 100 observers is wasteful even without a leak):

```

import { Injectable, DestroyRef, inject } from '@angular/core';

@Injectable() // scoped to the gallery/modal, not root, so it dies with the modal naturally
export class LazyLoadObserverService {
  private destroyRef = inject(DestroyRef);
  private callbacks = new Map<Element, () => void>();
  private observer = new IntersectionObserver(entries => {
    for (const entry of entries) {
      if (entry.isIntersecting) {
        this.callbacks.get(entry.target)?.();
        this.unwatch(entry.target);
      }
    }
  });

  constructor() {
    this.destroyRef.onDestroy(() => this.observer.disconnect());
  }

  watch(el: Element, onVisible: () => void) {
    this.callbacks.set(el, onVisible);
  }
}

```

```
this.observer.observe(e1);  
}  
  
unwatch(e1: Element) {  
  this.observer.unobserve(e1);  
  this.callbacks.delete(e1);  
}  
}
```

Verification: open/close the modal 10 times, heap-snapshot, and confirm `IntersectionObserver` instance count matches currently-active tiles (0 when modal is closed), not the cumulative ~1,000 created across all cycles.

Interviewer intent: Tests knowledge that modern observer-based Web APIs (Intersection/Resize/Mutation) have their own explicit disconnect lifecycle independent of DOM removal — a commonly-missed detail since these APIs feel more "passive" than `addEventListener`.

Scenario 17: @defer Block View Container Retaining Deferred Content After Repeated Toggling

Situation: A product page uses `@defer (on viewport)` to lazily load a heavy "Reviews" section. The section is inside a tab that users can switch away from and back to repeatedly (tab content is `*ngIf`-toggled, not router-based). After switching tabs 20 times, memory profiling shows the reviews chunk's components/data accumulating rather than being freed when the tab is hidden.

Question: Is this an `@defer` bug, or something about how the surrounding tab toggling interacts with it?

Answer: `@defer` itself is not the leak source — when the host `@if / *ngIf` (or the defer block's own trigger conditions going out of scope) causes Angular to destroy the enclosing view, the deferred content's component tree is destroyed along with everything else, exactly like any other Angular view. The far more likely explanation, consistent with the pattern across this chapter, is that the *deferred component itself* does something leak-prone internally (e.g., it renders a chart, opens a WebSocket for live review updates, or registers with a singleton service) — and the very fact that `@defer` lazily instantiates it means the leak is easy to miss in code review because the component's file feels "isolated" from the main bundle and gets less scrutiny.

A second, `@defer`-specific possibility worth checking: if the trigger is `on viewport` and the tab-toggle mechanism uses `[hidden] / CSS display: none` instead of actually removing the element from the DOM (a common mistake when building custom tab components for animation purposes), the element technically stays "in the viewport" per the `IntersectionObserver` `@defer` uses internally, so it may re-trigger or never properly unload — worth verifying the tab implementation uses `@if` / structural removal, not visibility toggling, if instant re-triggering or stale content is observed.

Diagnosis: Angular DevTools' component tree — switch away from the Reviews tab, inspect whether the deferred `ReviewsComponent` and its children still appear in the tree. If they've correctly disappeared, the leak is in whatever *that* component does internally (apply Scenarios 2/3/16's diagnostic approach directly to it) — heap snapshot search for the reviews component's constructor should show 0 after tab-away + GC if it's cleanly destroyed structurally.

Fix (assuming the underlying issue turns out to be, e.g., the reviews component opening a live-update WebSocket without cleanup — apply the pattern from Scenario 3):

```
@Component({
  selector: 'app-reviews',
  standalone: true,
  template: `...`,
})
export class ReviewsComponent {
  private destroyRef = inject(DestroyRef);

  constructor() {
    liveReviewUpdates$()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(update => this.applyUpdate(update));
  }

  private applyUpdate(update: unknown) { /* ... */ }
}
```

And confirm the tab container uses structural removal:

```
@if (activeTab() === 'reviews') {
  @defer (on viewport) {
    <app-reviews />
  } @placeholder {
    <div class="skeleton"></div>
  }
}
```

rather than `[hidden]="activeTab() !== 'reviews'"` wrapping the `@defer` block.

Verification: toggle tabs 20 times, confirm via Angular DevTools that `ReviewsComponent` is absent from the tree whenever the tab is inactive, and heap-snapshot to confirm the underlying leaky resource (WebSocket, chart, etc.) inside it is also cleaned up per the relevant scenario's fix.

Interviewer intent: Tests whether the candidate can correctly reason that `@defer` changes *when* a component is created, not whether it's destroyed properly, and won't misattribute an ordinary internal leak to the new deferred-loading feature just because it's new.

Scenario 18: Detached Angular Material Overlay/CDK Overlay Not Disposed

Situation: A custom autocomplete component built on Angular CDK's `overlay` service creates an overlay pane imperatively (`overlay.create(config)`, `overlayRef.attach(portal)`) to show suggestions, rather than using Material's built-in autocomplete. After typing in and closing the autocomplete field 50 times across a form, a heap snapshot shows 50 detached overlay pane `<div>` elements (class `cdk-overlay-pane`) still retained.

Question: What's the CDK Overlay-specific cleanup step being missed, and how do you make sure it always runs even if the user closes the field via unexpected paths (blur, Escape, clicking elsewhere)?

Answer: `overlayRef` requires an explicit `.dispose()` call — creating an overlay attaches a portal and inserts pane/backdrop elements directly into a shared CDK overlay container appended to `document.body`, entirely outside the component's own view/DOM subtree. Because it lives outside the component's template, Angular's normal view-destruction process does not touch it at all — it is 100% the developer's responsibility to call `overlayRef.dispose()`, and forgetting it leaves the pane element, its `ComponentPortal`'s attached component, and their event listeners (backdrop click, position strategy scroll listeners) all retained indefinitely.

The "even if closed via unexpected paths" part of the question is the crux of a robust fix: relying only on one code path (e.g., a `selectOption()` method that both selects and disposes) misses Escape-key dismissal, backdrop clicks, and the component itself being destroyed while the overlay is still open (e.g., user navigates away mid-search) — so dispose logic must be centralized and also hooked to `DestroyRef` as a safety net.

```
import { Component, inject, DestroyRef, ElementRef, viewChild } from '@angular/core';
import { Overlay, OverlayRef } from '@angular/cdk/overlay';
import { ComponentPortal } from '@angular/cdk/portal';
import { SuggestionsPanelComponent } from './suggestions-panel.component';

@Component({ selector: 'app-autocomplete', standalone: true, template: `<input #inputEl />` })
export class AutocompleteComponent {
  private overlay = inject(Overlay);
  private destroyRef = inject(DestroyRef);
  private inputEl = viewChild.required<ElementRef<HTMLInputElement>>('inputEl');
  private overlayRef?: OverlayRef;

  constructor() {
    // Safety net: guarantees disposal even if the component is destroyed
    // while the overlay is open (e.g., mid-search navigation away).
    this.destroyRef.onDestroy(() => this.closePanel());
  }

  openPanel() {
    if (this.overlayRef) return; // already open

    const positionStrategy = this.overlay.position()
      .flexibleConnectedTo(this.inputEl())
      .withPositions([
        { originX: 'start', originY: 'bottom', overlayX: 'start', overlayY: 'top' }
      ]);

    this.overlayRef = this.overlay.create({ positionStrategy, hasBackdrop: true });
    this.overlayRef.backdropClick().subscribe(() => this.closePanel()); // subscription dies
    // with overlayRef via dispose
    this.overlayRef.keydownEvents().subscribe(event => {
      if (event.key === 'Escape') this.closePanel();
    });
    this.overlayRef.attach(new ComponentPortal(SuggestionsPanelComponent));
  }

  closePanel() {
    if (this.overlayRef) {
      this.overlayRef.dispose();
    }
  }
}
```

```
this.overlayRef?.dispose(); // detaches portal, destroys attached component, removes
pane/backdrop DOM, clears its own internal listeners
this.overlayRef = undefined;
}
}
```

Note `overlayRef.dispose()` internally handles unsubscribing its own `backdropClick()` / `keydownEvents()` streams and destroying the attached portal's component, so a single centralized `closePanel()` covering all dismissal paths (blur, Escape, selection, component destroy) is sufficient — no need to manually track separate subscriptions for each.

Verification: open/close the autocomplete 50 times using a mix of dismissal paths (select an option, press Escape, click the backdrop, and — in a test — destroy the parent component mid-open), then heap-snapshot and search Summary view for "Detached" `cdk-overlay-pane` elements — count should be 0.

Interviewer intent: Tests specific knowledge that CDK Overlay content lives outside the Angular component tree by design (appended to `document.body`) and therefore requires manual `dispose()` — a very common real-world leak in apps with custom overlay-based UI (autocomplete, custom modals, tooltips).

Scenario 19: Web Worker Never Terminated

Situation: A CSV import feature spins up a `worker` to parse large files off the main thread, created fresh each time the user picks a file. After importing 15 different files in one session (without reloading), Chrome's Task Manager (`shift+Esc`) shows the tab's memory steadily climbing, and the OS-level process list shows multiple `worker` processes still present for a tab that should only have one active import at a time.

Question: How do you confirm worker instances are accumulating, and what's the correct termination pattern?

Answer: Web Workers are a good example of a leak that's *outside* the JS heap in the traditional sense — a running worker holds its own separate memory (its own global scope, any data it's parsing) that a main-thread heap snapshot won't fully capture. This is why Chrome Task Manager (which shows per-frame/per-worker memory, not just main-thread heap) is the right tool here, not just the Memory tab.

Diagnosis: `chrome://inspect/#workers` (or DevTools' "Sources" panel worker dropdown in newer Chrome) lists all active worker threads for the page — if this list grows with every file import instead of settling back down after each import completes, workers aren't being terminated. Cross-check with Chrome Task Manager's per-process memory column.

Root cause: the code creates `new worker(...)` and listens via `worker.onmessage`, but only ever calls `worker.terminate()` on the "happy path" (successful parse completion message), not on error, cancellation, or component destroy — so any import that errors out, or is abandoned by the user navigating away mid-import, leaves its worker running forever.

Fix — centralize termination so every exit path (success, error, component destroy) calls it exactly once:

```
import { Component, inject, DestroyRef, signal } from '@angular/core';

@Component({ selector: 'app-csv-importer', standalone: true, template: `...` })
export class CsvImporterComponent {
  private destroyRef = inject(DestroyRef);
```

```

private worker?: Worker;
progress = signal(0);

importFile(file: File) {
  this.cleanupworker(); // in case a previous import's worker is somehow still around

  this.worker = new Worker(new URL('./csv-parser.worker', import.meta.url));

  this.worker.onmessage = ({ data }) => {
    if (data.type === 'progress') {
      this.progress.set(data.value);
    } else if (data.type === 'done' || data.type === 'error') {
      this.cleanupworker(); // unified termination for BOTH success and error
    }
  };

  this.worker.onerror = () => this.cleanupworker(); // uncaught worker exceptions

  this.worker.postMessage({ file });

  this.destroyRef.onDestroy(() => this.cleanupworker()); // safety net for navigate-away
mid-import
}

private cleanupworker() {
  this.worker?.terminate();
  this.worker = undefined;
}
}

```

Verification: import 15 files, deliberately including some that error and one where you navigate away mid-import, then check `chrome://inspect/#workers` — the worker list should only ever show 0 or 1 entries for this feature (the currently-running import, if any), never an accumulating count, and Chrome Task Manager's memory for the tab should return near baseline between imports.

Interviewer intent: Tests whether the candidate knows worker memory is invisible to a standard main-thread heap snapshot and requires different tooling (Task Manager, `chrome://inspect/#workers`), and whether they think through all termination paths, not just the success path.

Scenario 20: Closures Over Large Data Captured by a Long-Lived Debounced Function

Situation: A form component creates a debounced autosave function (via a hand-rolled `debounce()` utility or `Lodash.debounce`) once in the constructor, closing over `this` to call `this.formValue` and `this.attachments` (an array that can hold large `File`/`Blob` objects for attached documents). Users fill out the form, attach large files, submit, and the form component is supposedly destroyed — but a heap snapshot shows the large `Blob` data still retained well after submission and navigation away.

Question: The component is destroyed and there's no obvious subscription or listener here — where's the reference actually coming from?

Answer: This scenario is specifically about closures capturing more than they need, not about a missing lifecycle hook — a good contrast question because the fix isn't "add takeUntilDestroyed," it's about *what the closure references*. `Lodash.debounce` (or a hand-rolled equivalent) returns a debounced function that internally holds a reference to the timer and the *most recent arguments/this-context* passed to it, so it can invoke the trailing call after the delay. If that debounced function is stored on a field of the component (e.g., `private debouncedSave = debounce(() => this.save(), 500)`), the debounced function itself is a closure over `this` (the whole component instance) — and critically, if anything *outside* the component holds a reference to that debounced function (e.g., it was passed to a shared root-level `AutosaveCoordinatorService` to be invoked centrally, or registered as a `window` event listener callback per Scenario 4/6's pattern), then the component — and everything it references, including `this.attachments` full of large Blobs — stays reachable indefinitely, since the component's own destruction doesn't matter if an external singleton still holds the closure.

Diagnosis: heap snapshot → search for large `Blob/ArrayBuffer` retained size entries in Summary view sorted by retained size descending → Reveal in Retainers on the largest one → trace the chain; it will likely read something like `AutosaveCoordinatorService._debouncedFns[] → (closure) → FormComponent → attachments[] → Blob`.

Fix — two complementary angles: (1) don't hand the closure to anything longer-lived than the component without also deregistering it on destroy (apply the Scenario 5 pattern), and (2) more fundamentally, avoid closing over large data unnecessarily — pass only what's needed at call time rather than capturing `this` broadly:

```
import { Component, inject, DestroyRef, signal } from '@angular/core';
import { debounce } from 'lodash-es';
import { AutosaveCoordinatorService } from './autosave-coordinator.service';

@Component({ selector: 'app-big-form', standalone: true, template: `...` })
export class BigFormComponent {
  private coordinator = inject(AutosaveCoordinatorService);
  private destroyRef = inject(DestroyRef);
  formValue = signal<Record<string, unknown>>({});
  attachments = signal<File[]>([]);

  // Debounced function closes over only the minimal data needed (a getter),
  // not `this` broadly, and is stored locally rather than registered elsewhere.
  private debouncedSave = debounce(() => {
    const snapshot = { ...this.formValue() }; // shallow copy of primitive/small data
    this.coordinator.save(snapshot);           // large attachments deliberately excluded
  }, 500);

  onFormChange() {
    this.debouncedSave();
  }

  constructor() {
    // Cancel any pending trailing invocation so it can't fire (and reference this
    // component) after destroy.
    this.destroyRef.onDestroy(() => this.debouncedSave.cancel());
  }
}
```

If the coordinator genuinely needs to hold a registered callback (e.g., to flush all pending autosaves on `beforeunload`), apply the register/unregister pattern from Scenario 5 so the reference is removed on destroy, in addition to `.cancel()`.

Verification: fill the form, attach large files, submit, navigate away, force GC, heap-snapshot, and check Summary view sorted by retained size — no `Blob` / `File` objects from this form should remain, and searching for `BigFormComponent` in the snapshot should show 0 retained instances with a clean (or absent) retainer chain.

Interviewer intent: Tests deeper understanding that closures can leak large data through debounce/throttle utilities specifically, and that the fix sometimes isn't a missing lifecycle hook at all but rather minimizing what a long-lived closure captures — a good question for distinguishing candidates who only know "add DestroyRef" as a reflex versus those who reason about closure scope.

Quick Revision Cheat Sheet

- **Angular destroying a component ≠ full cleanup.** `ngOnDestroy` / `DestroyRef` only tears down what Angular itself created; third-party libraries (charts, maps, tooltips, jQuery plugins) must be explicitly disposed via their own API (`.destroy()`, `.remove()`, `.dispose()`).
- `takeUntilDestroyed(destroyRef)` (or the implicit-injection-context form) is the default pattern for any RxJS subscription created inside a component/directive — pair it with `DestroyRef.onDestroy()` for anything that isn't an Observable (timers, sockets, listeners, third-party instances).
- **Singleton (`providedIn: 'root'`) services that hold references to components are the single most common real-world leak pattern.** Always pair `register()` with an `unregister()` / cleanup function invoked from the component's `DestroyRef`; prefer the component reading from the service (via signals/observables) over the service calling into the component.
- `Renderer2.listen()` returns an `unlisten` function — capture and call it. The same applies to any manual `addEventListener`: always store and pair with `removeEventListener`.
- `IntersectionObserver` / `ResizeObserver` / `MutationObserver` need explicit `.disconnect()` / `.unobserve()` — DOM removal alone does not stop them from being retained.
- **CDK Overlay, dynamically created components (`createComponent`), and portals live outside the Angular view tree** (often appended straight to `document.body`) — they require manual `.dispose()` / `.destroy()` / `ApplicationRef.detachView()` regardless of what happens to the "logical" parent component.
- **Web Workers and open WebSocket connections are resource leaks as well as memory leaks** — terminate/close them on every exit path (success, error, and component destroy), not just the happy path; worker memory is invisible to a main-thread heap snapshot, so use Chrome Task Manager / `chrome://inspect/#workers` instead.
- **Signals' `effect()` auto-ties to the enclosing `DestroyRef` unless you opt out with `manualCleanup: true` or a custom `injector`** — audit those opt-outs specifically when hunting effect-related leaks.
- **Detached DOM trees mean "reachable but not visible,"** not "still on screen" — a heap snapshot's "Detached" filter plus Retainers view is the fastest way to find the JS reference keeping a removed node alive.
- **Unbounded arrays/caches in long-lived services (logs, history, analytics buffers) are architectural**

leaks, not lifecycle bugs — fix with ring buffers/caps and by storing serialized data instead of live object/DOM references.

- **Route-reuse strategies and visibility-based ([hidden] /CSS) "removal" can silently prevent `ngOnDestroy` from ever running** — always confirm via Angular DevTools' component tree that a component actually left the tree before assuming its cleanup ran.
- **Debounce/throttle utilities capture closures over `this` and any data referenced at creation time** — minimize what they close over, and always `.cancel()` pending invocations on destroy so a queued call can't fire (and re-reference the destroyed component) later.
- **Core workflow for any suspected leak:** reproduce with a repeated action loop → Performance Monitor for a live trend → heap snapshot before/after with forced GC → Comparison view sorted by delta/retained size → Retainers view to find the actual holding reference → fix at the true root (often a singleton or third-party API, not the obviously-suspect component).

Created By - Durgesh Singh